contains()

| n | Vector | List | Hash | Tree |
|---|---|---|---|---|
| 1000 | 13.43 | 12.88 | 0 | 10.94 |
| 2000 | 16.76 | 18.41 | 0 | 19.24 |
| 4000 | 32.02 | 37.35 | 0 | 41.29 |
| 8000 | 69.97 | 75.24 | 0 | 82.82 |

query name

| n | Vector | List | Hash | Tree |
|---|---|---|---|---|
| 1000 | 146.5 | 140.2 | 141.6 | 87.3 |
| 2000 | 265.1 | 266.9 | 263.5 | 159.6 |
| 4000 | 670.8 | 647.7 | 630.2 | 431.6 |
| 8000 | 1298.4 | 1313.3 | 1339.2 | 922.5 |

query weight

| n | Vector | List | Hash | Tree |
|---|---|---|---|---|
| 1000 | 13.7 | 15.4 | 14.4 | 2.2 |
| 2000 | 28.3 | 31.1 | 29.4 | 3.7 |
| 56.7 | 56.7 | 65.0 | 65.9 | 7.2 |
| 8000 | 113.7 | 126.0 | 116.0 | 12.5 |

The results definitely match my expectations. The contains() was the fastest for the hash based inventory which took 0 time. This was most likely due to average constant time lookup. On the other hand, vector and list implementations showed linear growth throughout since it requires scanning through all elements. The AVL tree showed o(log) behavior as it was growing slower than the a linear growth but still increasing as the size increased.

For query,  the avl tree performed the best specifically in the weight. This is due to the fact that the tree maintains sorted order which allows for efficiency in range queries where it can search certain portions and avoid others. Vector, list, and hashes on the other hand, have to go through every element in linear time.

You would use a hash based inventory when using lookups with contains() as it is o(1). However, you would want to use a tree inventory when there are range queries that are required since it can retrieve the most relevant elements. The choice will ultimately depend on whether or not there is a priority in fast individual lookups or efficient ordered queries.